

Diagnóstico de Necessidade — Automação "Boletim IA"

Disciplina: AI Factory — Etapa 1 **Entrega:** Critério 1 (ID 1.1) **Autoria:** Emily Cristofoleti (individual) **Data:** 17/09/2026

1. Contexto

Estudante de Inteligência Artificial com necessidade contínua de acompanhar novidades, lançamentos e tendências do mercado. É uma área com características que agravam o problema:

- **Volume alto e diário** de publicações relevantes (novos modelos, papers, ferramentas, mudanças de API e de preço).
- **Ciclo de obsolescência curto:** o que era estado da arte há alguns meses deixa de ser referência de prática.
- **Fontes dispersas e heterogêneas:** comunidades técnicas, agregadores, blogs de empresas, repositórios e newsletters, cada uma com formato e sinal de qualidade diferentes.
- **Rotina concorrida:** estudo, projetos e trabalho consomem a maior parte do dia, e a atualização de mercado compete diretamente com esse tempo.

2. Problema

O processo de se manter atualizada é **manual, descentralizado e repetitivo:** abrir várias fontes, varrer títulos, decidir o que importa, ler o conteúdo, resumir mentalmente e, quando faz sentido, compartilhar. Não existe ponto único de entrada nem registro padronizado do que foi lido — a informação consumida hoje não fica recuperável amanhã.

3. Causas (por que o problema acontece)

1. **Ausência de agregação:** nenhuma das fontes acompanhadas reúne ou normaliza as outras; a integração é feita à mão, todos os dias.
2. **Curadoria 100% cognitiva:** é preciso ler para decidir o que vale a pena ler, o que torna o custo de atenção proporcional ao volume, não ao valor da informação.
3. **Sem memória do processo:** nada é consolidado em um histórico consultável, então não há como auditar o que foi acompanhado nem medir evolução.
4. **Ferramentas de leitura não destilam:** leitores de RSS e feeds entregam tudo cru; o trabalho de síntese continua sendo humano.
5. **Barreira de entrada para automatizar:** a maioria dos caminhos triviais passa por chaves de API pagas, serviços em nuvem ou TTS de terceiros — custo recorrente e dependência externa.

4. Consequências (impacto de não resolver)

- **Fadiga cognitiva** e consequente abandono do hábito de acompanhar; a informação acaba "voltando a ser novidade" semanas depois.
- **Atraso na absorção**: decisões de estudo e de projeto partem de um retrato desatualizado do mercado.
- **Trabalho repetido**: a mesma filtragem manual é refeita todos os dias, tempo que deixa de ser investido em prática e aprofundamento.
- **Conhecimento não auditável**: sem registro, não é possível reconstruir quando uma tecnologia apareceu, o que mudou e o que já tinha sido lido.
- **Risco de informação rasa ou incorreta**: sem critério consistente de curadoria, títulos chamativos e conteúdo especulativo competem de igual para igual com fontes confiáveis.
- **Custo oportunista**: manter-se atualizada deixa de ser um ativo da formação e passa a ser uma dívida de tempo.

5. Oportunidade

Automatizar o ciclo completo de curadoria com **IA local**, sem custo recorrente e sem chave de API:

1. **Coleta automática** de notícias recentes de IA em fontes públicas e anônimas.
2. **Curadoria e síntese** por um LLM local, que transforma manchetes dispersas em um roteiro coeso em português.
3. **Síntese de voz** do roteiro, gerando um boletim em áudio consumível em deslocamento e em tarefas paralelas.
4. **Persistência** de cada edição em um banco de dados simples, criando histórico auditável.

A oportunidade une duas frentes da formação: resolve uma necessidade real de rotina e, ao mesmo tempo, exercita o encadeamento completo de um pipeline de IA (coleta → LLM → TTS → persistência) com tecnologia local.

6. Solução proposta

Automação **Boletim IA**, implementada como pipeline Python de quatro etapas orquestradas por um único comando (`python main.py`):

Etapa	Módulo	O que faz
Coleta	<code>src/coletor.py</code>	Consulta o Hacker News (API Algolia) e o Reddit (feed RSS Atom) e normaliza as notícias em <code>fonte</code> , <code>titulo</code> , <code>url</code> , <code>texto_base</code> , <code>data_publicacao</code> e <code>score</code>
Curadoria/resumo	<code>src/boletim.py</code>	Envia as notícias ao Ollama (qwen2.5:3b) com prompt de âncora de podcast e regra anti-alucinação, produzindo um roteiro em pt-BR
Síntese de voz	<code>src/audio.py</code>	Gera o áudio do boletim com Kokoro ONNX (voz pt-BR <code>pf_dora</code>), em WAV PCM 16 bits / 24 kHz
Persistência	<code>src/registro.py</code>	Registra cada edição na tabela <code>boletins</code> do SQLite (<code>boletim.db</code>)

7. Ganhos esperados

- **Eficiência:** uma única execução substitui a varredura manual de múltiplas fontes, eliminando a filtragem repetitiva diária.
- **Consumo assíncrono:** o boletim em áudio pode ser ouvido em deslocamento e em tarefas paralelas, deixando de competir com o tempo de estudo.
- **Custo zero de operação:** 100% local e offline após o download dos modelos — sem chaves de API, sem nuvem, sem assinatura.
- **Histórico auditável:** cada edição fica persistida e consultável por data.
- **Reaproveitamento:** o pipeline é modular e extensível (roadmap: orquestração em n8n, enriquecimento por scraping, aumento do tamanho do modelo).

8. Verificação da oportunidade (critério de sucesso)

O protótipo é considerado bem-sucedido quando um único comando gera, de ponta a ponta e sem intervenção manual, três evidências observáveis:

1. Arquivo de áudio do boletim (`audio/boletim_<data>.wav`) íntegro.
2. Roteiro em português produzido pelo LLM a partir de notícias reais do dia.
3. Registro da edição na tabela `boletins` (`boletim.db`), recuperável por consulta SQL.

Resultado obtido: execução E2E real em 17/09/2026 gerou 20 notícias coletadas, roteiro de 3.185 caracteres, áudio de 201,33 s (9.663.850 bytes, 24 kHz, PCM 16 bits) e registro `id=1` no banco — validado por consulta SQL direta. O áudio dessa execução foi preservado como `audio/boletim_2026-09-17_primeira_execucao.wav`, porque o pipeline foi rodado novamente no mesmo dia e o nome do arquivo (que usa a data) foi sobrescrito. A segunda execução, completa e independente, gerou o registro `id=2` (121,07 s), confirmando a reprodutibilidade do pipeline.